

**PROJECT REPORT**

**FITBUDDY**

# **AI Fitness Plan Generator using Gemini Models**

A Web-Based Artificial Intelligence Application

Submitted as a Project Report.

Technology Stack: Python • FastAPI • Google Gemini • SQLite • SQLAlchemy • Jinja2

Academic Year 2026

# 1. Abstract

FitBuddy is a web-based application designed to generate personalized fitness plans with the support of Google Gemini models. The system accepts user information such as age, weight, fitness goal, and preferred workout intensity and uses these inputs to produce a structured seven-day fitness plan.

The project combines a FastAPI backend, HTML/Jinja2 interface, Gemini AI services, and SQLite database storage through SQLAlchemy. In addition to initial plan generation, the application supports feedback so that a user can request an updated plan. Nutrition and recovery guidance can also be generated as a separate AI-assisted feature.

The main purpose of FitBuddy is to simplify the creation of structured fitness routines through a web interface while keeping user and plan information organized. The project demonstrates how a Python web framework, database layer, template-based frontend, and generative AI service can work together in a single application.

# 2. Introduction

Fitness planning often requires organizing goals, workout intensity, daily activities, and recovery suggestions into a usable routine. FitBuddy approaches this problem as an application-development project in which artificial intelligence assists with generating a structured plan from user-provided information.

The system is intended to make plan creation more convenient through a simple browser interface. Instead of manually constructing a weekly schedule, the user enters profile information and a goal, after which the backend prepares an AI request and displays the generated result.

The application also includes a feedback cycle. A user can submit feedback about a generated plan, and the system can request an updated plan from the AI layer. This creates an interaction model in which the output can be refined rather than treated as a fixed first response.

# 3. Problem Statement

Creating a personalized weekly fitness routine manually can require time and repeated organization of user information, exercise preferences, and goals. A basic static fitness webpage may display general advice but does not provide an interactive process for generating a plan from individual inputs.

FitBuddy addresses this application-level problem by providing a structured input form, an AI-assisted generation process, persistent storage, feedback submission, and result rendering through a single web application.

## 4. Objectives

The major objective is to develop a web application that generates a personalized seven-day fitness plan using Gemini models.

Specific objectives include collecting relevant user information; validating inputs through a FastAPI backend; sending structured prompts to Gemini; displaying the generated plan in a readable format; storing users and plans using SQLite and SQLAlchemy; accepting feedback; generating updated plans; and providing nutrition or recovery tips.

## 5. Existing System and Proposed System

The existing approach represented in the project context is based mainly on manually prepared or generic fitness routines. Such routines may not provide an application-level mechanism for taking user inputs and generating a structured weekly response.

The proposed FitBuddy system introduces an interactive web workflow. User details are entered through the frontend, processed by FastAPI, passed to Gemini for generation, stored in the database, and displayed through Jinja2 templates. Feedback can be used to request a revised plan.

| Aspect           | Existing Approach          | FitBuddy Approach                   |
|------------------|----------------------------|-------------------------------------|
| Plan creation    | Manual or generic routine  | AI-assisted personalized generation |
| User input       | Limited                    | Age, weight, goal, intensity        |
| Weekly structure | Prepared separately        | Seven-day plan generated by AI      |
| Feedback         | May be manual              | Feedback route supports plan update |
| Storage          | Not necessarily integrated | SQLite with SQLAlchemy              |
| Interface        | Static or separate tools   | FastAPI + Jinja2 web interface      |

## 6. Scope of the Project

- Provide a browser-based interface for entering fitness-related information.
- Generate a seven-day workout plan through Gemini.
- Provide nutrition or recovery tips through an AI request.
- Allow feedback submission and plan updating.
- Store user and plan information using SQLite and SQLAlchemy.
- Provide an admin/coach-style view for users and plans.
- The uploaded project material does not specify a production medical or clinical decision-support scope; therefore, this report treats FitBuddy as an educational web application for fitness-plan generation.

## 7. Requirements Analysis

### 7.1 Functional Requirements

- User profile input: age, weight, goal, and workout intensity.
- Workout generation through the backend.
- AI-generated seven-day plan display.
- Nutrition/recovery tip generation.
- Feedback submission.
- Updated plan generation based on feedback.
- Storage and retrieval of users and plans.
- Administrative view of stored users and plans.

### 7.2 Non-Functional Requirements

- Simple and understandable web interface.
- Fast response handling through asynchronous web architecture where appropriate.
- Maintainable Python backend structure.
- Persistent database storage.
- Clear separation between frontend, backend, AI service, and database.

## 8. Feasibility Study

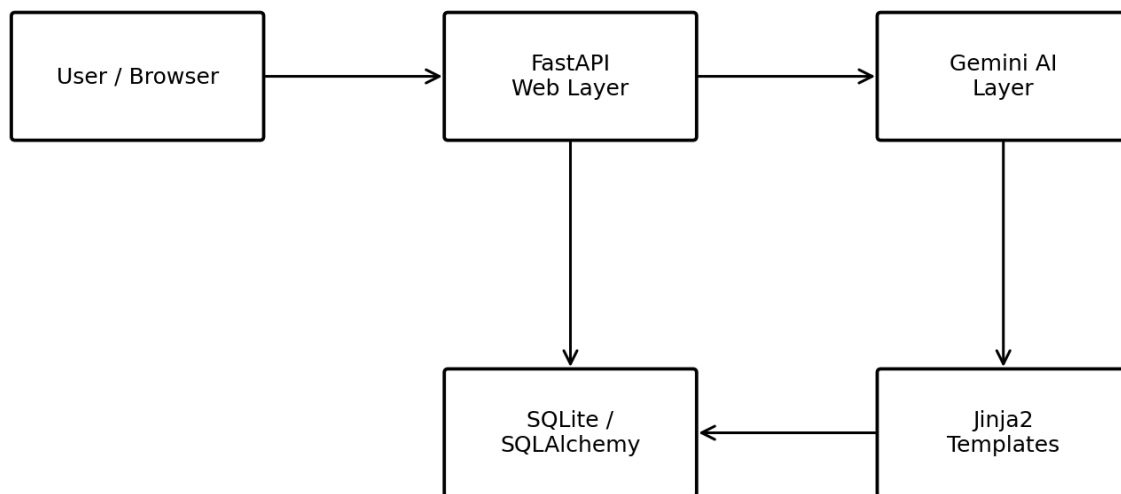
| Feasibility Type | Observation                                                                                                                   |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Technical        | Python, FastAPI, Gemini API, SQLite, SQLAlchemy and Jinja2 provide the required application components.                       |
| Operational      | The browser workflow is straightforward: enter information, generate a plan, view the result, and optionally submit feedback. |
| Economic         | The project uses common development tools and a lightweight local database; API usage is dependent                            |

|          |                                                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------------------------------|
|          | on the selected Gemini service and account configuration.                                                                   |
| Schedule | The work can be divided into architecture, backend, AI integration, frontend, database, testing, and deployment milestones. |

## 9. System Architecture

FitBuddy follows a layered architecture. The frontend is implemented with HTML/Jinja2 templates. FastAPI handles routes and application logic. Gemini models form the AI layer for workout and nutrition generation. SQLite provides persistent storage through SQLAlchemy.

FitBuddy System Architecture



*Figure 1: Newly designed FitBuddy system architecture.*

# 10. Project Workflow

The overall workflow begins with user information collection and continues through validation, AI generation, database storage, and result display. Feedback introduces a second generation cycle for plan refinement.

FitBuddy End-to-End Workflow

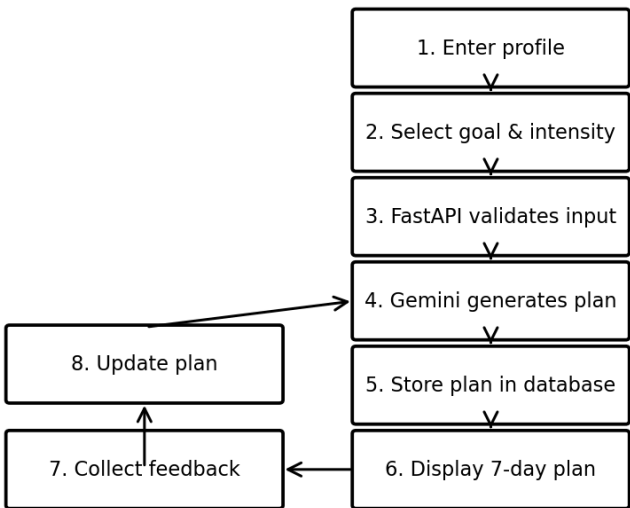
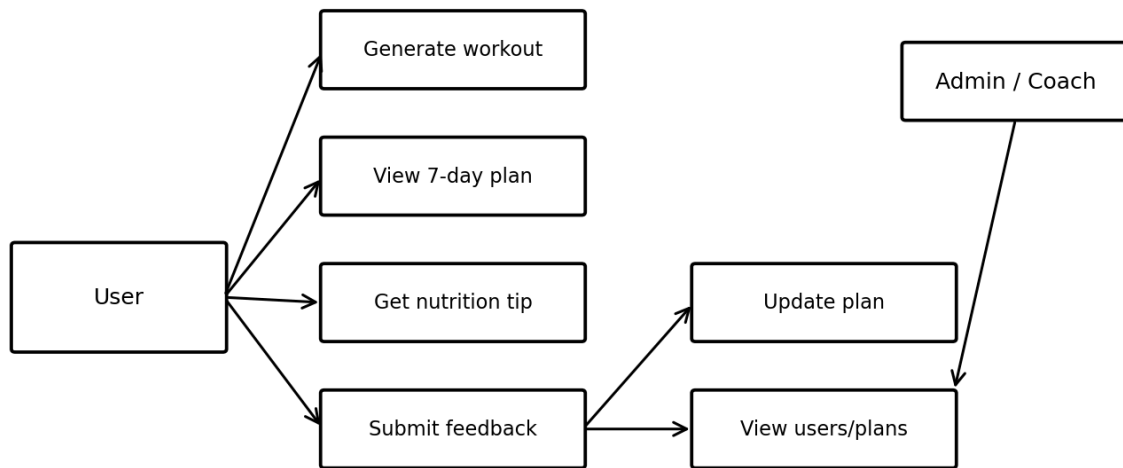


Figure 2: Newly designed end-to-end FitBuddy workflow.

# 11. Use-Case Analysis

The primary user interacts with the application to generate a workout plan, view results, request nutrition guidance, and submit feedback. The admin/coach-oriented function provides access to stored users and plans.

## FitBuddy Use-Case Diagram



*Figure 3: Newly designed FitBuddy use-case diagram.*

## 12. Data Flow and Interaction Design

### 12.1 Level-0 Data Flow

| Stage | Input          | Processing                   | Output                   |
|-------|----------------|------------------------------|--------------------------|
| 1     | User details   | Validate and prepare request | Structured profile       |
| 2     | Profile + goal | Create Gemini prompt         | AI request               |
| 3     | AI request     | Gemini generates content     | Workout/nutrition result |
| 4     | Generated plan | Store through SQLAlchemy     | Database record          |
| 5     | Stored result  | Render Jinja2 template       | Browser display          |
| 6     | Feedback       | Create update request        | Revised plan             |

### 12.2 User Interaction

1. Open the FitBuddy home page.
2. Enter the required profile information.
3. Choose the fitness goal and preferred intensity.
4. Submit the generation request.
5. Receive and view the seven-day plan.
6. Optionally request nutrition/recovery guidance.
7. Submit feedback when changes are required.
8. View the updated result.

## 13. Database Design

The project uses SQLite as the storage layer and SQLAlchemy for database interaction. The model described by the source material supports user and plan storage, while the feedback flow allows information related to plan updates to be processed.

## FitBuddy Entity Relationship Model

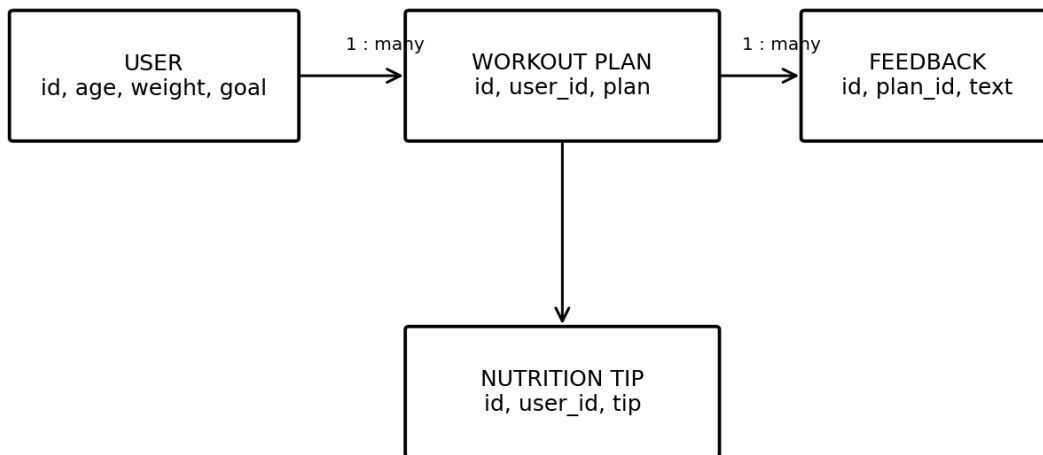


Figure 4: Newly designed conceptual entity relationship model.

| Entity        | Representative Information                     |
|---------------|------------------------------------------------|
| User          | User identity and profile information          |
| Workout Plan  | Generated plan associated with a user          |
| Feedback      | Feedback associated with a plan/update request |
| Nutrition Tip | AI-generated nutrition or recovery guidance    |

## 14. AI Integration Design

Gemini is used as the generative AI layer. The source project describes Gemini Pro for workout generation and feedback-based updates, and Gemini Flash for nutrition tips. The application prepares user information as part of the request and renders the returned result in the web interface.

### 14.1 AI Processing Steps

- Collect validated user inputs.
- Construct a task-specific prompt.
- Send the request to the configured Gemini model.
- Receive the generated response.
- Process the response for display.
- Store relevant plan information where required.
- Return the result to the user interface.

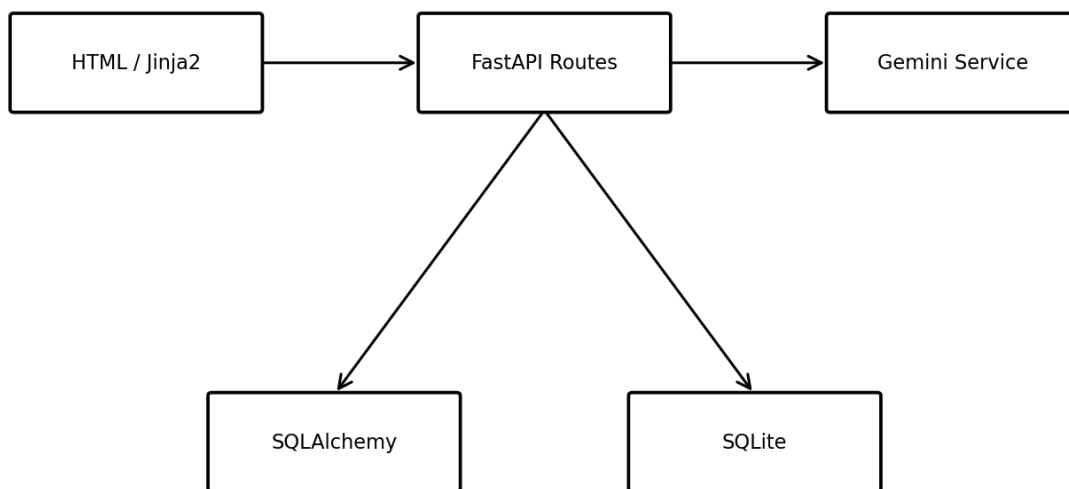
## 14.2 Prompt Design Principles

- Clearly identify the user's goal.
- Include the selected workout intensity.
- Request a seven-day structure.
- Keep the requested response organized for web display.
- Use feedback as additional context when generating an update.

## 15. Component Design

The application can be understood as a set of cooperating components: presentation templates, FastAPI route handlers, AI service interaction, SQLAlchemy data access, and SQLite storage.

### FitBuddy Component Design



*Figure 5: Newly designed FitBuddy component diagram.*

# 16. Sequence Diagram

The sequence below illustrates the interaction among the user, FastAPI backend, Gemini service, and database during plan generation.

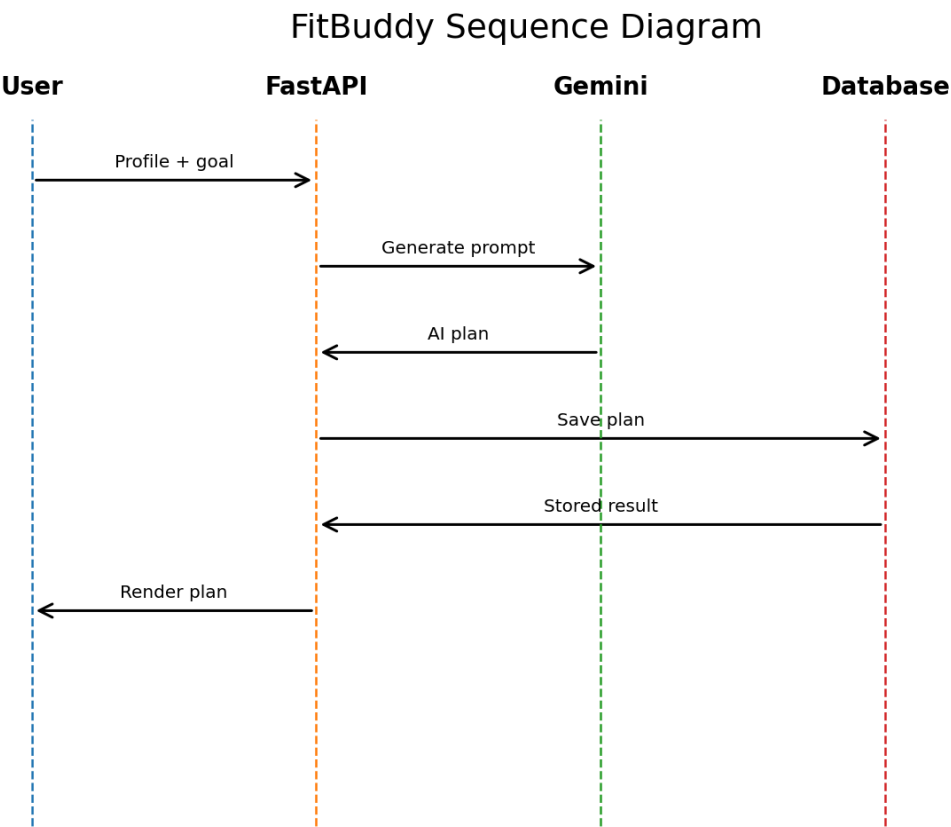


Figure 6: Newly designed FitBuddy sequence diagram.

# 17. Backend Development

## 17.1 FastAPI Role

FastAPI acts as the central backend framework. It receives browser requests, validates input, coordinates Gemini calls, performs database operations, and returns rendered pages.

## 17.2 Main Routes

| Route             | Purpose                                                           |
|-------------------|-------------------------------------------------------------------|
| /                 | Displays the main input interface.                                |
| /generate-workout | Accepts user inputs and generates the workout plan.               |
| /submit-feedback  | Processes user feedback and supports plan updating.               |
| /view-all-users   | Provides the admin/coach-oriented view of stored users and plans. |

## 17.3 Processing Logic

The route layer follows a request-processing pattern: receive form data, validate it, prepare the AI request, call the relevant Gemini service, store or retrieve information as needed, and render the appropriate Jinja2 template.

# 18. Frontend Development

The frontend uses HTML, CSS, and Jinja2 templates. The project material identifies pages such as index.html for the main interface, result.html for generated results, and all\_users.html for the administrative view.

## 18.1 Main Interface

- User input fields for profile information.
- Goal and intensity selection.
- Generation action.
- Clear result presentation.

## 18.2 Result Interface

- Seven-day workout information.
- Nutrition or recovery tip where requested.
- Feedback entry for plan refinement.

## 18.3 Responsive Design

The interface is intended to remain understandable across common browser screen sizes. Layout, spacing, and readable result sections are important because AI-generated content may be longer than a simple static page. Sure — here is a more detailed, project-report style version you can use:

The frontend is the visible part of the application that allows users to interact with the system. It is developed using HTML, CSS, and Jinja2 templates. HTML is used to create the structure of the web pages, CSS is used for styling and layout, and Jinja2 is used to dynamically display information generated by the backend.

The project contains different pages for user interaction and administration, such as index.html for the main interface, result.html for displaying generated workout plans, and all\_users.html for the administrative view.

## 18.1 Main Interface

The main interface acts as the starting point for the user.

It provides input fields for entering profile details such as age, gender, fitness level, or other required information.

Users can select their fitness goal, such as weight management, muscle building, or general fitness.

Users can choose the required workout intensity according to their preference or fitness level.

A Generate button is provided to submit the information and request a personalized plan.

The interface is designed to be simple and easy to understand.

Proper labels and input fields help users provide accurate information.

Validation can be used to ensure that required fields are not left empty.

## 18.2 Result Interface

The result interface displays the workout plan generated by the system.

It presents a seven-day workout schedule in an organized format.

Each day can contain information about the exercise, duration, repetitions, or intensity.

A nutrition or recovery tip can also be displayed when requested.

The generated content is separated into clear sections so that users can easily follow the plan.

A feedback section allows users to provide their comments or suggestions.

User feedback can be used to improve or refine future workout plans.

Jinja2 templates help display the generated information dynamically without creating a separate static page for every possible result.

## 18.3 Responsive Design

Responsive design ensures that the application remains usable on different devices and screen sizes.

The layout adjusts according to the browser or device screen size.

Proper spacing and alignment improve readability.

Text and buttons should remain clearly visible on both desktop and mobile screens.

Workout information is divided into readable sections to prevent the page from becoming difficult to understand.

Since AI-generated responses may contain a large amount of text, the design provides sufficient spacing and structured presentation.

Responsive design improves the overall user experience and accessibility of the application.

#### 18.4 Administrative Interface

The `all_users.html` page provides an administrative view of registered or interacting users.

It can display relevant user information collected by the application.

Data can be presented in a structured table or list format.

The page helps administrators monitor the users and information stored by the system.

Proper formatting makes large amounts of user information easier to view.

#### 18.5 Role of Jinja2

Jinja2 is used to connect the frontend templates with the backend application.

It allows backend-generated data to be inserted into HTML pages.

Dynamic information such as the user's name, workout schedule, tips, and feedback can be displayed automatically.

It reduces the need to create separate HTML pages for every user.

It helps create reusable and maintainable templates.

## 19. Development Environment and Setup

### 19.1 Prerequisites

- Python environment.
- FastAPI and Uvicorn.
- Google Gemini API access/configuration.
- HTML/CSS/Jinja2.
- SQLAlchemy and SQLite.
- Git for source management.
- Virtualenv or conda for environment isolation.

### 19.2 Suggested Project Structure

| Folder/File             | Purpose                           |
|-------------------------|-----------------------------------|
| <code>app/</code>       | Application source package        |
| <code>templates/</code> | Jinja2 HTML templates             |
| <code>static/</code>    | CSS and static frontend resources |
| <code>routes.py</code>  | FastAPI route definitions         |

|                  |                               |
|------------------|-------------------------------|
| models.py        | Database model definitions    |
| database.py      | Database configuration/access |
| services/        | AI/service integration logic  |
| requirements.txt | Python dependencies           |

### 19.3 Local Execution

9. Create and activate a Python virtual environment.
10. Install the required dependencies.
11. Configure the Gemini API credentials according to the development environment.
12. Start the FastAPI application using Uvicorn.
13. Open the application in a browser.
14. Use the API documentation endpoint when testing backend routes.

## 20. Core Functional Modules

| Module            | Function                                            |
|-------------------|-----------------------------------------------------|
| Profile Module    | Collects age, weight, goal and intensity.           |
| Workout Generator | Requests a structured seven-day plan from Gemini.   |
| Nutrition Module  | Produces nutrition/recovery guidance.               |
| Feedback Module   | Accepts feedback and triggers an update request.    |
| Storage Module    | Persists users and plans through SQLAlchemy/SQLite. |
| Admin View        | Displays stored users and plans.                    |

## 21. Algorithm / Processing Logic

The core algorithm is an application workflow rather than a mathematical optimization algorithm. Its purpose is to transform structured user input into an AI request and then persist and display the response.

15. Start application.
16. Receive profile information.
17. Check required fields and values.
18. Identify goal and workout intensity.
19. Build the workout-generation prompt.
20. Send the request to Gemini.
21. Receive the generated seven-day plan.
22. Save the plan and related information.
23. Render the result page.
24. Accept optional feedback.
25. If feedback is provided, create an update prompt.
26. Generate and display the revised plan.

## 22. Testing Strategy

### 22.1 Functional Testing

| Test Case | Expected Result |
|-----------|-----------------|
|-----------|-----------------|

|                          |                                            |
|--------------------------|--------------------------------------------|
| Valid profile submitted  | Workout generation request is processed.   |
| Missing required input   | Validation prevents incomplete processing. |
| Valid goal and intensity | Request is passed to AI layer.             |
| Successful AI response   | Plan is displayed.                         |
| Feedback submitted       | Update request is processed.               |
| User/plan retrieval      | Stored information is shown in admin view. |

## 22.2 Interface Testing

- Verify form labels and controls.
- Verify result sections are readable.
- Verify navigation between input, result, feedback, and admin pages.
- Verify generated content does not break the page layout.

## 22.3 Integration Testing

Integration testing should verify the complete chain from browser request to FastAPI route, Gemini request, database storage, and final HTML response. The same process should be checked for the feedback-based update flow.

# 23. Error Handling, Security and Privacy

## 23.1 Error Handling

- Validate required user inputs before making an AI request.
- Handle unavailable or failed AI responses gracefully.
- Handle database errors without exposing internal implementation details.
- Provide understandable messages for invalid requests.

## 23.2 Security Considerations

- Keep API credentials outside source code where possible.
- Validate user-provided input.
- Restrict administrative routes appropriately in a production deployment.
- Use secure configuration for deployed applications.
- Store only information required for the application's purpose.

## 23.3 Privacy Considerations

Because the application processes user profile information, the deployment should communicate what information is collected and how it is used. The supplied project description establishes the storage of user and plan information but does not specify a complete privacy policy; a production system would require one.

# 24. Deployment and Maintenance

The application can be tested locally with Uvicorn and accessed through the browser. FastAPI also provides an API documentation interface that can assist development and route testing. For deployment, the application

requires a Python runtime, configured dependencies, database availability, and secure Gemini API configuration.

## 24.1 Maintenance Tasks

- Update dependencies when required.
- Monitor API integration changes.
- Review database records and backups.
- Test routes after code changes.
- Improve prompt templates based on observed output quality.
- Maintain frontend templates and styling.

## 25. Limitations

- AI-generated content depends on the quality and availability of the configured Gemini service.
- Generated plans are application outputs and should not be treated as professional medical advice.
- The source project does not describe a clinical validation process.
- SQLite is appropriate for the project scope but may require a different database architecture for larger deployments.
- The report does not claim that AI generation guarantees safe or optimal exercise selection for every individual.

## 26. Future Enhancements

- Add user authentication and role-based access.
- Add richer progress tracking.
- Add calendar-style workout presentation.
- Provide more structured exercise metadata.
- Add configurable plan duration.
- Add more detailed feedback categories.
- Introduce stronger production database infrastructure when required.
- Improve monitoring and logging for deployed environments.
- Add additional AI-assisted personalization while retaining validation and safety controls.

## 27. Results and Discussion

The completed design demonstrates how the major technologies can be connected into a single application workflow. FastAPI provides the web service layer, Gemini provides generative capabilities, SQLAlchemy provides structured database access, SQLite provides local persistence, and Jinja2 connects backend results to HTML pages.

The main functional result is a personalized seven-day plan generated from user-provided information. The feedback mechanism extends the interaction by allowing a new generation request using user comments. The admin/coach view provides a basic way to inspect stored application data.

| Feature                         | Project Result                             |
|---------------------------------|--------------------------------------------|
| Personalized workout generation | Supported through Gemini integration       |
| Seven-day plan                  | Core output                                |
| Nutrition/recovery tip          | Supported AI-assisted feature              |
| Feedback update                 | Supported workflow                         |
| Persistent storage              | SQLite + SQLAlchemy                        |
| Web interface                   | FastAPI + HTML/Jinja2                      |
| Admin/coach view                | Supported through dedicated route/template |

## 28. Conclusion

FitBuddy demonstrates a practical integration of generative AI with a Python web application. The project combines structured user input, FastAPI routes, Gemini model interaction, database storage, Jinja2 templates, and feedback-based plan updating into one workflow.

The design provides a clear foundation for an academic AI application project. Its modular structure also makes it possible to extend the application with authentication, progress tracking, richer data models, improved interfaces, and additional AI-assisted functions in future versions.

## 29. References and Development Resources

- FastAPI documentation and framework concepts used for backend route development.
- Google Gemini API/model documentation for AI service integration.
- SQLAlchemy documentation for Python database interaction.
- SQLite documentation for lightweight relational storage.
- Jinja2 documentation for server-side HTML templating.
- Uvicorn documentation for running the FastAPI application.

Note: The project-specific architecture, workflow, routes, features, and technical scope in this report are based on the uploaded FitBuddy project example. External documentation is listed here as development resources rather than as claims about additional implemented features.

## 30. Appendix

### Appendix A – Sample User Input

| Field  | Example Value                                |
|--------|----------------------------------------------|
| Age    | User-entered age                             |
| Weight | User-entered weight                          |
| Goal   | Weight loss / muscle gain / general wellness |

|                   |                                            |
|-------------------|--------------------------------------------|
| Workout Intensity | Selected intensity level                   |
| Feedback          | Optional comments about the generated plan |

## Appendix B – Sample Generation Flow

Input → Validation → Prompt Construction → Gemini Request → Generated Plan → Database Storage → Result Display → Feedback → Updated Plan

## Appendix C – Project Deliverables

- FastAPI backend.
- Jinja2-based frontend templates.
- Gemini AI integration.
- SQLite/SQLAlchemy persistence.
- Workout generation workflow.
- Nutrition/recovery tip workflow.
- Feedback-based update workflow.
- Admin/coach-oriented user and plan view.
- Testing and deployment documentation.

Prepared as a new academic project report based on the uploaded FitBuddy example, with rewritten content and newly designed diagrams.